

Binary Trees, Part II

Rebuilding trees from traversals, saving them as strings, and walking them with zero extra memory.

What's inside

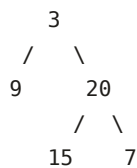
1. What each traversal actually tells you
2. Build from Preorder + Inorder
3. Build from Postorder + Inorder
4. Serialize & Deserialize
5. Morris Traversal ($O(1)$ space)
6. Revision sheet

How to read these notes

Every topic follows the same rhythm: *the one observation that makes it click*, a picture, a small dry run by hand, then the code. If you only remember one thing per topic, remember the yellow box.

0 What each traversal actually tells you

Our running example for the whole handout:



Traversal	Order	Sequence	Where is the root?
Preorder	Root, L, R	3 9 20 15 7	first element
Inorder	L, Root, R	9 3 15 20 7	somewhere in the middle
Postorder	L, R, Root	9 15 7 20 3	last element

THE CORE OBSERVATION

Preorder and postorder answer “**who is the root?**”

Inorder answers “**who goes left and who goes right?**”

In inorder, the root is printed *after* its entire left subtree and *before* its entire right subtree. So once you find the root's position in inorder, everything to its left *is* the left subtree and everything to its right *is* the right subtree. Inorder is a ruler; the root is where you cut.

```

inorder:    9   | 3   | 15 20 7
            ----|----|-----
            left  root  right
  
```

Why preorder + postorder is not enough

Look at these two different trees:

Tree A	Tree B	
1	1	
/	\	
2	2	
pre : 1 2	pre : 1 2	<- same
post: 2 1	post: 2 1	<- same
in : 2 1	in : 1 2	<- different!

Pre and post both say “1 is the root and 2 is below it”, but neither says *which side*. Only inorder breaks the tie. That’s why every construction problem pairs something with **inorder**. (Pre + post *does* work if the tree is *full*, i.e. every node has 0 or 2 children, but that’s a special case.)

Assumption for sections 1 and 2: all values are distinct. We look up “where is value x in inorder”, and that lookup only makes sense if x appears once.

All code in these notes uses this node:

```
struct Node {
    int val;
    Node *left, *right;
    Node(int x) : val(x), left(nullptr), right(nullptr) {}
};
```

1 Build from Preorder + Inorder

THE CORE OBSERVATION

Preorder hands you roots *in the exact order you need them*: root, then the whole left subtree, then the whole right subtree. Inorder tells you how big each side is.

So: take the next preorder value as the root, find it in inorder, split, and recurse **left first**, then right.

THE PICTURE

```
preorder: [ 3 ][ 9 ][ 20 15 7 ]
           root L      R

inorder : [ 9 ][ 3 ][ 15 20 7 ]
           L  root  R
```

The left block of inorder has 1 element, so the left block of preorder also has 1 element. Same set of nodes, just listed in a different order. Recurse on each block and the tree falls out.

WHY LEFT MUST BE BUILT BEFORE RIGHT

We keep a single pointer `preIdx` that walks preorder from left to right. Preorder lists the left subtree *before* the right one, so the values the pointer meets next belong to the left side. If you built right first, the right subtree would steal the left subtree’s roots.

DRY RUN

`build(lo, hi)` builds the tree for the inorder range `[lo..hi]`. Inorder positions: $9 \rightarrow 0$, $3 \rightarrow 1$, $15 \rightarrow 2$, $20 \rightarrow 3$, $7 \rightarrow 4$.

call	preIdx	root	mid	left range	right range
<code>build(0,4)</code>	0	3	1	<code>[0,0]</code>	<code>[2,4]</code>
<code>build(0,0)</code>	1	9	0	<code>[0,-1]</code> empty	<code>[1,0]</code> empty
<code>build(2,4)</code>	2	20	3	<code>[2,2]</code>	<code>[4,4]</code>
<code>build(2,2)</code>	3	15	2	empty	empty
<code>build(4,4)</code>	4	7	4	empty	empty

Notice `preIdx` goes 0, 1, 2, 3, 4 with no jumping around. Preorder is consumed exactly in order, one value per node.

CODE

```
unordered_map<int,int> pos; // value -> index in inorder
int preIdx;

Node* build(vector<int>& pre, int lo, int hi) {
    if (lo > hi) return nullptr; // empty range = no node
    Node* root = new Node(pre[preIdx++]); // next preorder value is the root
    int mid = pos[root->val]; // where it sits in inorder
    root->left = build(pre, lo, mid - 1); // LEFT first (preorder order)
    root->right = build(pre, mid + 1, hi);
    return root;
}

Node* buildTree(vector<int>& pre, vector<int>& in) {
    pos.clear();
    for (int i = 0; i < (int)in.size(); i++) pos[in[i]] = i;
    preIdx = 0;
    return build(pre, 0, (int)in.size() - 1);
}
```

COST Time $O(n)$ · Space $O(n)$

Each node is created once and each lookup is $O(1)$ thanks to the map. Space is the map plus the recursion stack, which is $O(h)$: $O(\log n)$ for a balanced tree, $O(n)$ for a skewed one.

ONE-LINE TRICK Preorder picks the root, inorder cuts the array. Walk preorder forward, build left then right.

WHERE PEOPLE SLIP

- ✗ Searching inorder with a loop every time. It still works, but it's $O(n^2)$ and TLEs on a skewed tree of 10^5 nodes. Use the map.
- ✗ Building right before left. Preorder is Root-L-R; the pointer must follow that order.
- ✗ Forgetting to reset `preIdx = 0` between test cases (it's a global).
- ✗ Base case written as `lo == hi`. The correct empty check is `lo > hi`; a range with one element still needs a node.
- ✗ Duplicate values. The map stores one index per value, so the split becomes wrong.

Aside: the version without a global pointer

Interviewers sometimes want it passed explicitly. The only thing to remember is the size of the left part:

```
// preorder range [pL..pR], inorder range [iL..iR]
int mid = pos[pre[pL]];
int leftSize = mid - iL;
root->left = go(pL + 1, pL + leftSize, iL, mid - 1);
root->right = go(pL + leftSize + 1, pR, mid + 1, iR);
```

2 Build from Postorder + Inorder

THE CORE OBSERVATION

The root of postorder is the **last** element. So walk postorder **from the back**.

Postorder is L, R, Root. Read it backwards and you get **Root, R, L**. Right after the root, the next values you meet belong to the *right* subtree. That's the whole reason we build **right first**.

THE PICTURE

```
postorder:      [ 9 ][ 15 7 20 ][ 3 ]
                  L      R      root

read it backwards: [ 3 ][ 20 7 15 ][ 9 ]
                    root      R      L      <- right comes first

inorder:         [ 9 ][ 3 ][ 15 20 7 ]
                    L  root      R
```

Think of it as a mirror image of section 1. Preorder is Root-L-R; reversed postorder is Root-R-L. Same algorithm, pointer moves the other way, children swapped.

DRY RUN

Same inorder positions as before. `postIdx` starts at 4 and moves *down*.

call	postIdx	root	mid	right range (first)	left range (second)
<code>build(0,4)</code>	4	3	1	[2,4]	[0,0]
<code>build(2,4)</code>	3	20	3	[4,4]	[2,2]
<code>build(4,4)</code>	2	7	4	empty	empty
<code>build(2,2)</code>	1	15	2	empty	empty
<code>build(0,0)</code>	0	9	0	empty	empty

WHAT GOES WRONG IF YOU BUILD LEFT FIRST

After creating 3, we'd call `build(0,0)` for the left side and grab `post[3] = 20`. But 20 isn't even in the range `[0,0]`; the only node there is 9. The pointer handed a right-subtree root to the left subtree, and every node after that is wrong. The order of recursion has to match the order the pointer reads.

CODE

```
unordered_map<int,int> pos;
int postIdx;

Node* build(vector<int>& post, int lo, int hi) {
    if (lo > hi) return nullptr;
    Node* root = new Node(post[postIdx--]); // read postorder from the back
    int mid = pos[root->val];
    root->right = build(post, mid + 1, hi); // RIGHT first!
    root->left = build(post, lo, mid - 1);
    return root;
}

Node* buildTree(vector<int>& in, vector<int>& post) {
    pos.clear();
    for (int i = 0; i < (int)in.size(); i++) pos[in[i]] = i;
    postIdx = (int)post.size() - 1;
    return build(post, 0, (int)in.size() - 1);
}
```

COST Time $O(n)$ · Space $O(n)$

Identical reasoning to section 1.

ONE-LINE TRICK Postorder backwards is Root, Right, Left. So pointer from the end, build right before left.

WHERE PEOPLE SLIP

- ✗ Copy-pasting the preorder solution and only changing `++` to `--`. You must also swap the two recursive calls.
- ✗ Starting `postIdx` at 0. The root lives at the end.
- ✗ On LeetCode 106 the parameter order is (inorder, postorder), not the other way round. Read the signature.

Side by side

	Pre + In	Post + In
Root comes from	pre[preIdx++]	post[postIdx--]
Pointer starts at	0	$n - 1$
Recurse order	left, then right	right , then left
Why	pre = Root L R	reversed post = Root R L
Inorder's job	split into L and R	split into L and R

3 Serialize & Deserialize

The task. Turn a tree into a string, and later turn that string back into *exactly* the same tree: same values, same shape.

THE CORE OBSERVATION

A plain traversal loses the shape because you can't tell where a subtree *ends*. Fix it by writing a marker, say #, for every NULL child.

Now every subtree is self-contained: it starts with its root and stops exactly when both of its children have been closed off with # (or with their own finished subtrees). With markers, **preorder alone is enough**; no inorder needed.

WHY THE MARKERS MATTER

Take Tree A and Tree B from section 0 again.

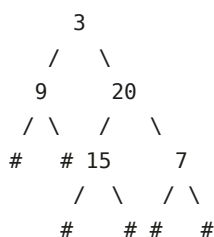


without #: "1 2" "1 2" <- can't tell them apart
 with # : "1 2 # # #" "1 # 2 # #" <- different strings

In Tree B the # right after 1 says "1 has no left child", so 2 must be on the right. The marker records the shape that the values alone can't.

SERIALIZE THE RUNNING EXAMPLE (PREORDER + #)

Draw the tree with every empty slot shown:



Preorder over this picture gives:

3 9 # # 20 15 # # 7 # #

Quick check: 5 nodes and 6 #s. A tree with n nodes always has $n + 1$ null slots (there are $2n$ child slots, and $n - 1$ of them are used by real nodes, because everyone except the root has a parent).

DESERIALIZE: DRY RUN

Read tokens one at a time from the front. Each call to `des()` eats exactly one token:

- if it's #, return NULL;
- otherwise make a node, then call `des()` for its left child, then for its right child.

Because the left call keeps eating tokens until the left subtree is complete, the right call automatically starts at the right place. Nobody has to count anything.

token	action	what we're filling
3	new node 3	root
9	new node 9	3.left
#	NULL	9.left
#	NULL (9 is finished)	9.right
20	new node 20	3.right
15	new node 15	20.left
#	NULL	15.left
#	NULL (15 is finished)	15.right
7	new node 7	20.right
#	NULL	7.left
#	NULL (7, 20, 3 all finished)	7.right

CODE (PREORDER WITH #)

```
// ----- serialize -----
void ser(Node* root, string& s) {
    if (!root) { s += "# "; return; }           // mark the empty slot
    s += to_string(root->val) + " ";           // Root
    ser(root->left, s);                          // Left
    ser(root->right, s);                        // Right
}
string serialize(Node* root) { string s; ser(root, s); return s; }

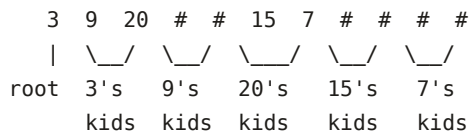
// ----- deserialize -----
Node* des(istream& in) {
    string tok;
    in >> tok;                                // eat exactly one token
    if (tok == "#") return nullptr;
    Node* root = new Node(stoi(tok));
    root->left = des(in);                       // same order as serialize
    root->right = des(in);
    return root;
}
Node* deserialize(const string& data) {
    istringstream in(data);
    return des(in);
}
```

COST Time $O(n)$ · Space $O(n)$

Every node and every # is written and read once. The string is $O(n)$ and the recursion depth is $O(h)$.

Level-order version (the LeetCode format)

Same idea, but we write nodes level by level using a queue. When deserializing, each node we pop from the queue reads the *next two* tokens: its left child, then its right child.



```

string serialize(Node* root) {
    string s;
    queue<Node*> q; q.push(root);
    while (!q.empty()) {
        Node* cur = q.front(); q.pop();
        if (!cur) { s += "# "; continue; }
        s += to_string(cur->val) + " ";
        q.push(cur->left);           // push even if NULL
        q.push(cur->right);
    }
    return s;
}

Node* deserialize(const string& data) {
    istringstream in(data);
    string tok; in >> tok;
    if (tok == "#") return nullptr;
    Node* root = new Node(stoi(tok));
    queue<Node*> q; q.push(root);
    while (!q.empty()) {
        Node* cur = q.front(); q.pop();
        in >> tok;           // left child of cur
        if (tok != "#") { cur->left = new Node(stoi(tok)); q.push(cur->left); }
        in >> tok;           // right child of cur
        if (tok != "#") { cur->right = new Node(stoi(tok)); q.push(cur->right); }
    }
    return root;
}

```

Also $O(n)$ time and $O(n)$ space. The BFS version avoids deep recursion, which matters for a skewed tree with 10^5 nodes.

ONE-LINE TRICK # marks where a subtree ends. Deserialize in the *same order* you serialized, one token per call.

WHERE PEOPLE SLIP

- x** No separator between values. "12" could be twelve or one-then-two. Always use a space or comma.
- x** Storing each value as a single char. Breaks on 10, -5, 1000. Use `to_string` / `stoi`.
- x** Serializing preorder but deserializing in a different order (e.g. building right first). The reading order must mirror the writing order.
- x** Skipping # for leaf children to "save space". That throws away exactly the information that makes the shape unique.
- x** In the BFS version, only pushing non-null children while serializing. Then the #s never get written.

4 Morris Traversal

The task. Print the inorder traversal using **$O(1)$ extra space**: no recursion, no stack.

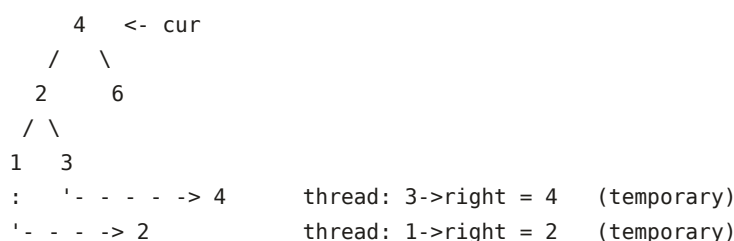
FIRST, WHY DO WE NORMALLY NEED A STACK?

Inorder goes left first. When the left subtree is done, we have to get *back up* to the parent to print it. Tree pointers only point down, so the stack (or recursion) is our memory of "where to return".

THE CORE OBSERVATION

Who is the *last* node printed in *cur*'s left subtree? Its **inorder predecessor**: the rightmost node of the left subtree. That node's right pointer is NULL, i.e. wasted.

So before going left, point that wasted right back at *cur*. It's a temporary "return ticket". When the traversal later follows it, we're back at *cur* with no stack, and we tear the ticket up so the tree is restored.

THE PICTURE

Node 3 is the rightmost node of 4's left subtree, so it's 4's predecessor. Node 1 is 2's predecessor.

THE THREE CASES (THIS IS THE WHOLE ALGORITHM)

1. **No left child.** Nothing to do on the left. **Visit cur**, go `cur = cur->right`.
(That right pointer might be a real child *or* a thread going back up. Either way, follow it.)
2. **Left child exists, predecessor's right is NULL.** First time at *cur*. Make the thread `pre->right = cur`, then go left.
3. **Left child exists, predecessor's right is cur.** We just came back up through our own thread, so the left side is finished. Remove the thread, **visit cur**, go right.

To find the predecessor: step once left, then keep going right *while* right is not NULL and not cur.

DRY RUN ON THE TREE ABOVE

cur	pre	case	action	output
4	3	2	thread 3→4, go left to 2	
2	1	2	thread 1→2, go left to 1	
1	–	1	visit 1, follow right (thread) to 2	1
2	1	3	remove 1→2, visit 2, go right to 3	1 2
3	–	1	visit 3, follow right (thread) to 4	1 2 3
4	3	3	remove 3→4, visit 4, go right to 6	1 2 3 4
6	–	1	visit 6, go right to NULL: stop	1 2 3 4 6

Every thread that was made got removed, so the tree ends up exactly as it started.

CODE: INORDER

```
vector<int> morrisInorder(Node* root) {
    vector<int> res;
    Node* cur = root;
    while (cur) {
        if (!cur->left) {                // case 1
            res.push_back(cur->val);
            cur = cur->right;
        } else {
            Node* pre = cur->left;        // find inorder predecessor
            while (pre->right && pre->right != cur) pre = pre->right;

            if (!pre->right) {            // case 2: first visit
                pre->right = cur;        // make the thread
                cur = cur->left;
            } else {                    // case 3: came back
                pre->right = nullptr;    // remove the thread
                res.push_back(cur->val);
                cur = cur->right;
            }
        }
    }
    return res;
}
```

PREORDER IS A ONE-LINE MOVE

In preorder you print the root *before* its left subtree. The first time we meet cur is case 2, so print it there instead of in case 3:

```
if (!pre->right) {                    // case 2
    res.push_back(cur->val);          // <- visit here for PREORDER
    pre->right = cur;
    cur = cur->left;
} else {                            // case 3
    pre->right = nullptr;            // no visit here
    cur = cur->right;
}
```

Case 1 stays the same in both versions.

COST Time $O(n)$ · Space $O(1)$

The predecessor search looks expensive, but each edge on a “left, then all the way right” path is walked at most twice: once to create the thread and once to remove it. Add it up over the whole tree and it’s still $O(n)$. Morris postorder exists too but is fiddly (it reverses right-spine paths); inorder and preorder are what get asked.

ONE-LINE TRICK Borrow the predecessor’s empty right pointer as a way back. Thread on the way down, cut on the way back.

WHERE PEOPLE SLIP

- ✗ Predecessor loop written as `while (pre->right)`. Once a thread exists, that loop runs into `cur` and around forever. You need `&& pre->right != cur`.
- ✗ Forgetting `pre->right = nullptr` in case 3. Output looks fine, but you’ve left a cycle inside the tree.
- ✗ Returning early (“find the k-th smallest, then return”) while threads are still in place. The caller gets back a corrupted tree. Either finish the loop or accept that the tree is modified.
- ✗ Visiting in case 2 when you wanted inorder (that gives preorder), or in case 3 when you wanted preorder.
- ✗ Using it on a shared or read-only tree in a multithreaded setting. Morris temporarily *writes* to the tree.

5 Revision sheet

Topic	Remember this	Cost
Pre + In	Preorder picks the root, inorder cuts the array. Pointer forward, left then right.	$O(n)$ time, $O(n)$ space
Post + In	Postorder backwards = Root, R, L. Pointer from the end, right then left.	$O(n)$ time, $O(n)$ space
Pre + Post	Not unique in general (Tree A vs Tree B). Only works for full binary trees.	–
Serialize	Write # for every NULL. Preorder + # alone fixes the shape. n nodes $\Rightarrow n + 1$ #.	$O(n)$ time, $O(n)$ space
Deserialize	One token per call, same order as you wrote. # returns NULL.	$O(n)$ time, $O(n)$ space
Morris	Predecessor's NULL right becomes a thread back to cur. Make it, follow it, remove it.	$O(n)$ time, $O(1)$ space

The three traversals in one breath

Preorder	Root L R	-> root is FIRST	-> "who is the root"
Inorder	L Root R	-> root is INSIDE	-> "what's left, what's right"
Postorder	L R Root	-> root is LAST	-> "who is the root" (read backwards)

Morris in three lines

no left	-> visit, go right	
left, pre->right=NULL	-> make thread, go left	(preorder visits here)
left, pre->right=cur	-> cut thread, visit, go right	(inorder visits here)

Quick self-test

1. Preorder 1 2 4 5 3, inorder 4 2 5 1 3. Draw the tree.
2. Inorder 4 2 5 1 3, postorder 4 5 2 3 1. Draw the tree and check it matches Q1.
3. Serialize the tree from Q1 with preorder + #. How many # should you get?
4. Deserialize "1 # 2 3 # # #". What shape is it?
5. Run Morris inorder on the tree from Q1. List every thread you create, in order.

Answers. 1–2: root 1; 1.left = 2 (children 4, 5); 1.right = 3. 3: 1 2 4 # # 5 # # 3 # #, six #. 4: 1 has no left child; 1.right = 2; 2.left = 3. 5: threads 5→1, then 4→2; output 4 2 5 1 3.